

Project 2: Classification

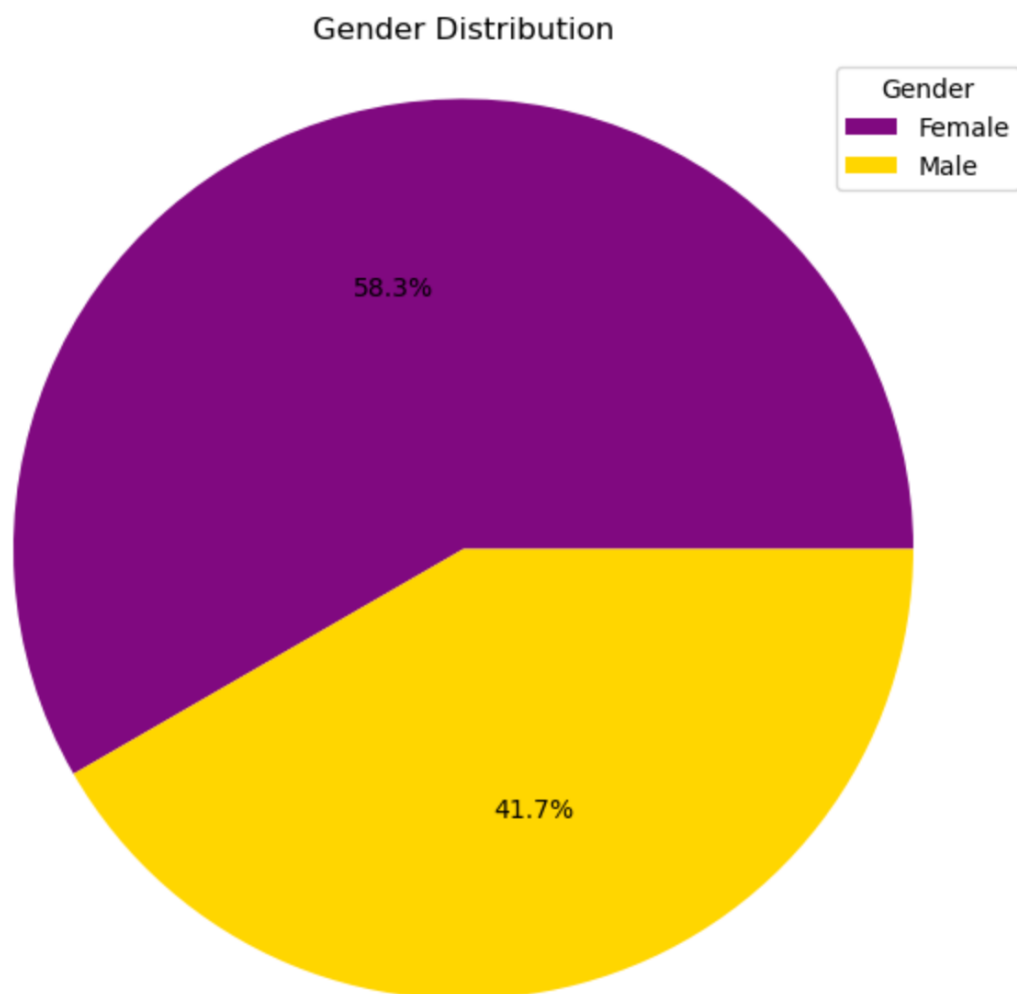
Steffi Dorothy

10/5/2024

Instructor name: Roe Shraga

Task 2 - EDA

Distribution Of Gender



The pie chart shows the gender distribution in the dataset.

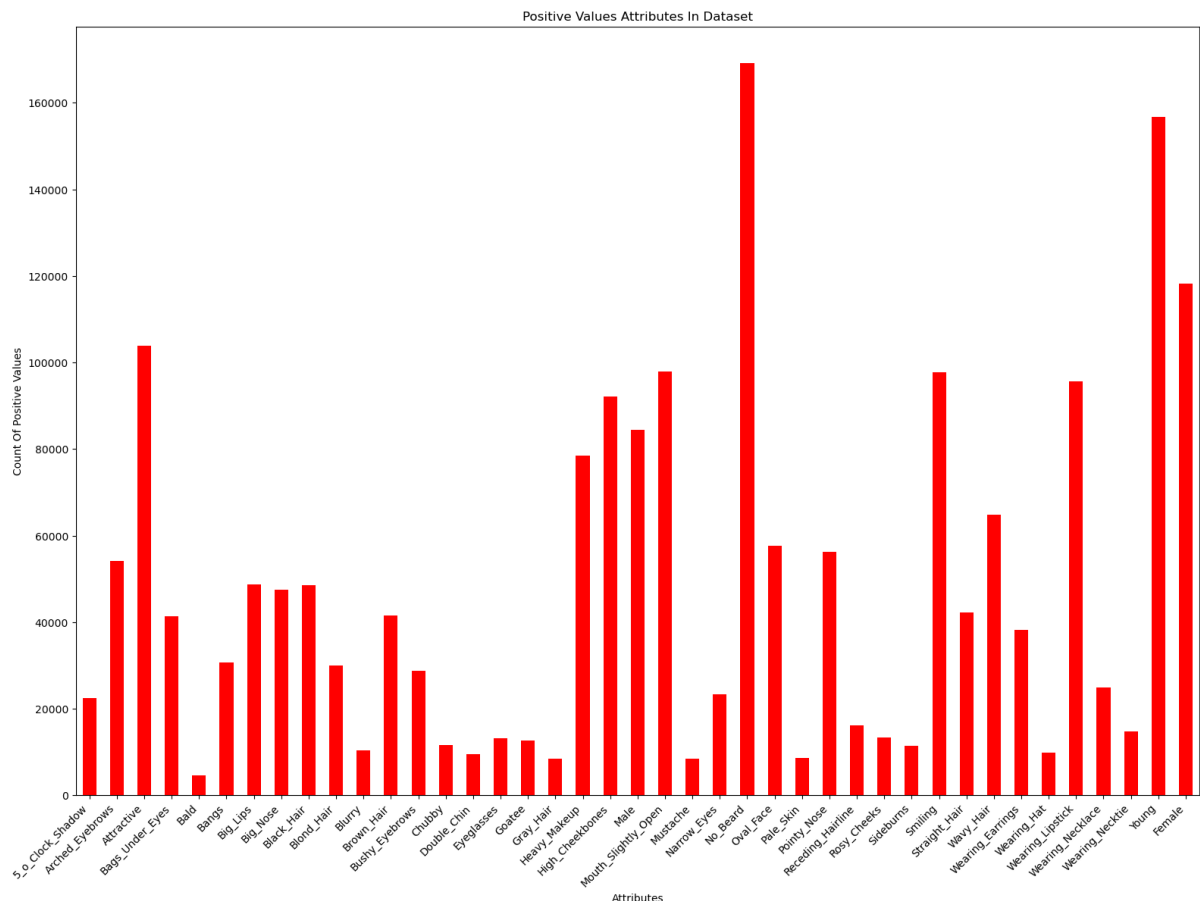
The results are as follows:

- Female: 58.3%

- Male: 41.7%

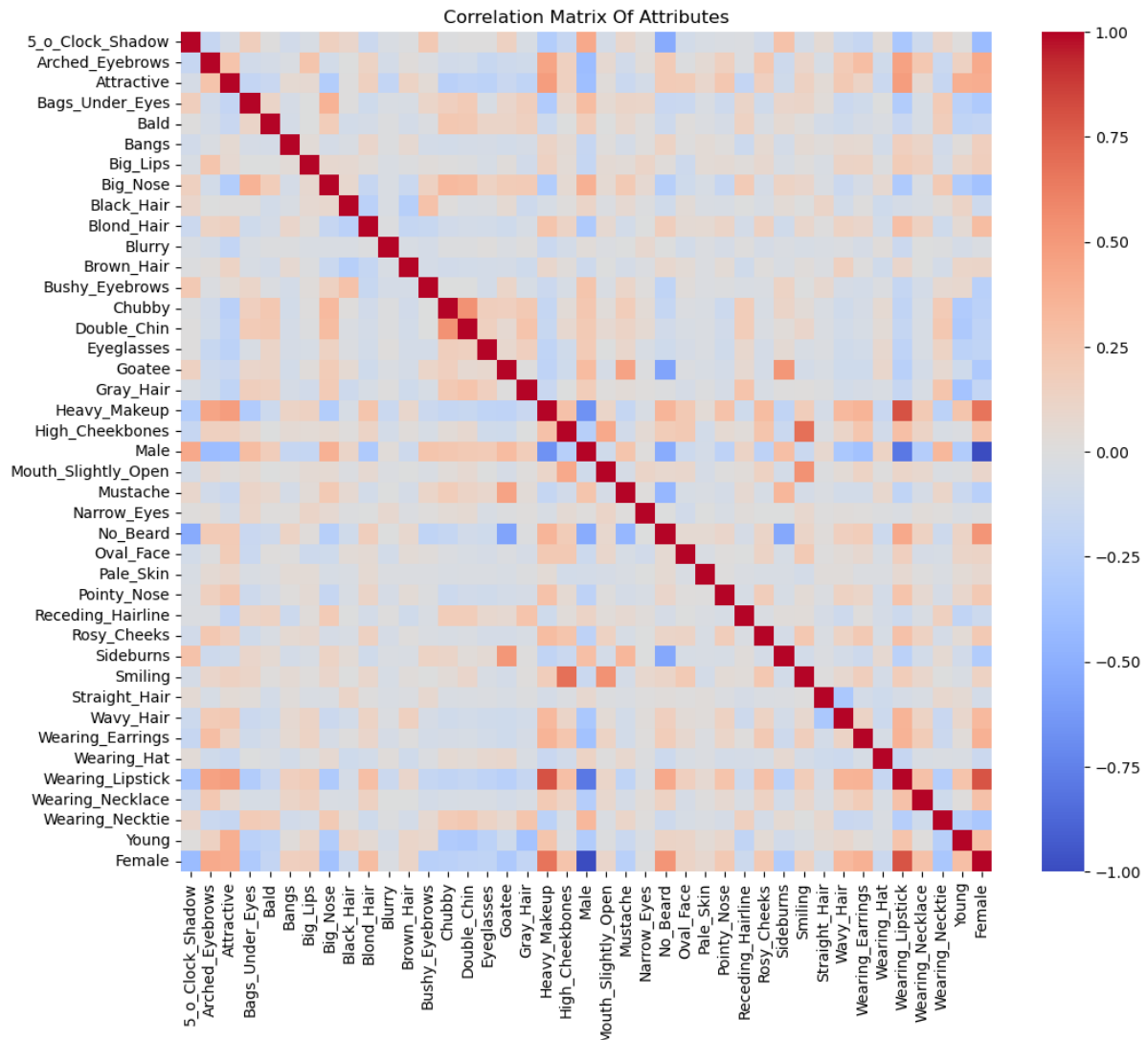
“ This indicates that the dataset is slightly skewed towards female individuals. ”

Distribution Of Attributes



This dataset is about celebrities, including many famous actors around the world. As a result, the "Young" and "Attractive" columns have high counts. The dataset features females, and not every male has facial hair, which explains why the "No Beard" column has the highest count among the male attributes.

Correlation Matrix of Attributes



The correlation matrix shows the relationships between various facial attributes.

Key Observations:

- **Diagonal Line:** The diagonal line running from the top left to the bottom right shows the correlation of each attribute with itself, which is always 1.0.
- Red indicates a strong positive correlation. Attributes with a high positive correlation tend to occur together.
- Blue indicates a strong negative correlation. Attributes with a high negative correlation tend to exclude each other.
- White indicates a weak or no correlation.

Specific Relationships (Based on the Visible Correlations):

- **Gender:**

- Male has a strong negative correlation with Young and Female, suggesting that older individuals are more likely to be male.
- Male has a strong positive correlation with No_Beard, indicating that men are less likely to have beards.
- Facial Hair:
 - Beard and Mustache show a strong positive correlation, suggesting that individuals with beards are more likely to have Mustaches.
 - Sideburns have a moderate positive correlation with Beard, suggesting a connection between these facial hair attributes.
- Age:
 - Young has a strong negative correlation with Bald, Gray_Hair, and Receding_Hairline, indicating that younger individuals are less likely to have these attributes.
- Appearance:
 - Attractive has a moderate positive correlation with High_Cheekbones, suggesting a connection between these attributes.
 - Wearing_Lipstick has a strong positive correlation with Females, indicating that lipstick is more commonly associated with women.

Limitations:

- The correlation matrix only shows linear relationships between attributes.
- Correlation does not imply causation. A strong correlation between two attributes does not necessarily mean that one causes the other.

Scatter plot matrix with density plots

It visualizes the relationships between various facial attributes.

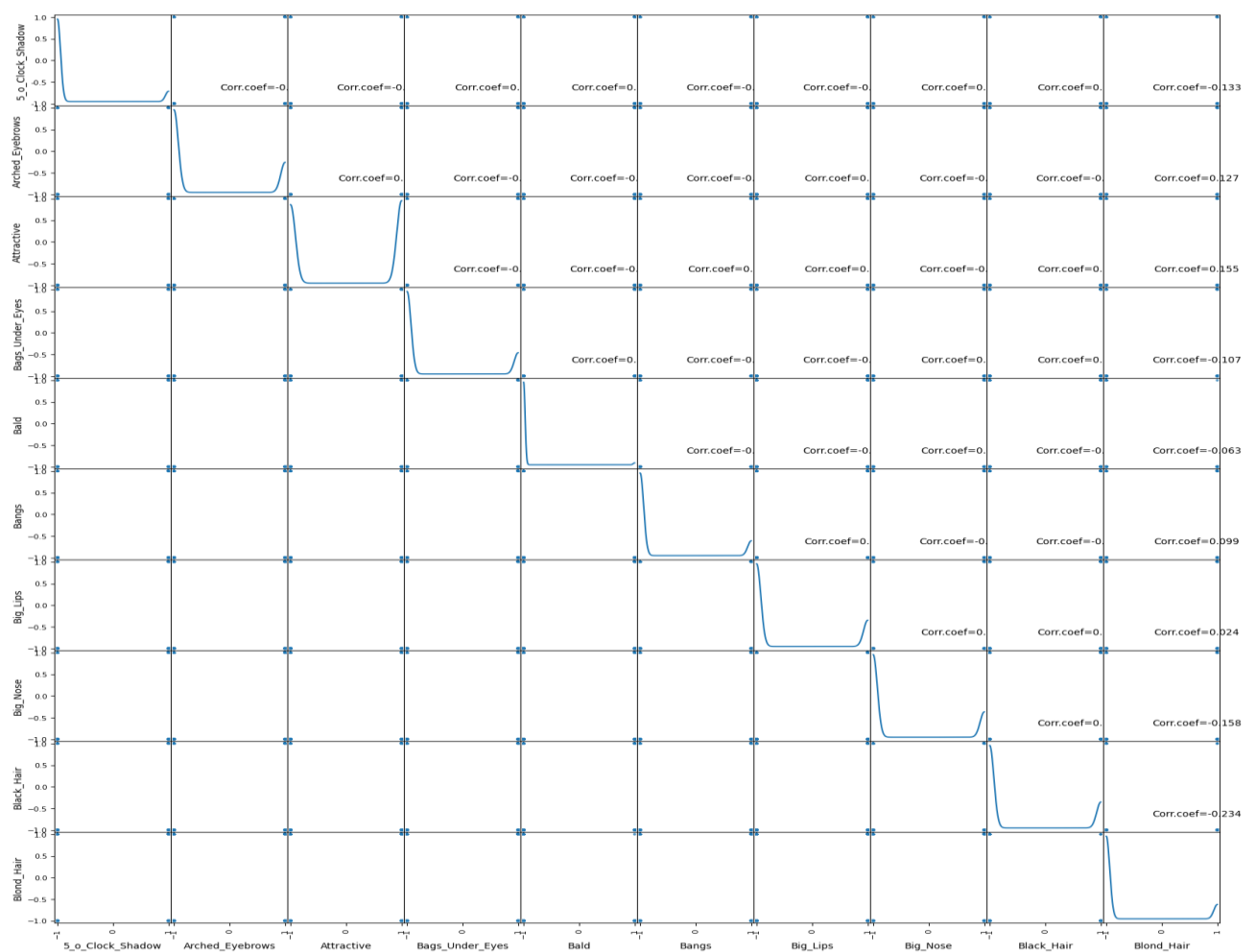
Key Observations:

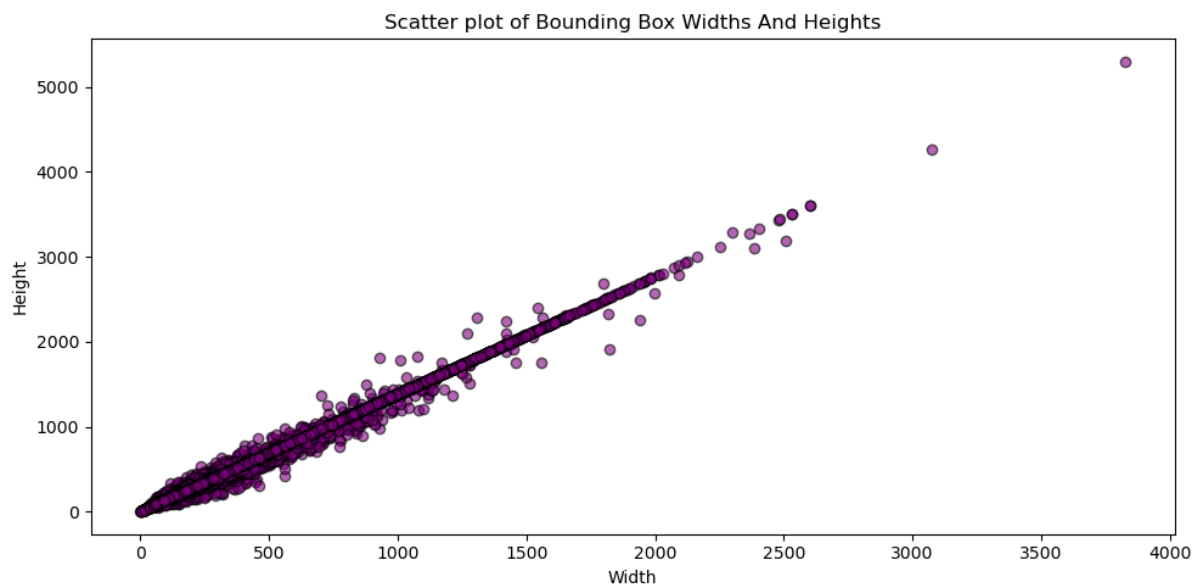
- Scatter Plots: Each subplot represents a pair of attributes. The scatter plot within each subplot shows the distribution of data points for those attributes.
- Density Plots: The marginal plots along the top and right edges of the matrix are density plots. They show the distribution of each attribute.
- Correlation Coefficients: The numbers displayed in each subplot are the correlation coefficients between the corresponding attributes. These values range from -1 to 1, indicating the strength and direction of the linear relationship.
- Attributes: The attributes represented in the matrix include facial features like 5 o'clock shadow, arched eyebrows, attractiveness, bags under eyes, baldness, bangs, big lips, big nose, black hair, blonde hair, and more.

Inferences (Based on Visual Observations):

- Attractiveness: There seems to be a weak positive correlation between attractiveness and having arched eyebrows or big lips.
- Hair: Blonde hair appears to be less common compared to black hair.
- Facial Features: There are no strong correlations observed between facial features like 5 o'clock shadow, bags under eyes, and baldness.

Scatter And Density Plot





The scatter plot shows the relationship between the widths and heights of a set of bounding boxes.

Key Observations:

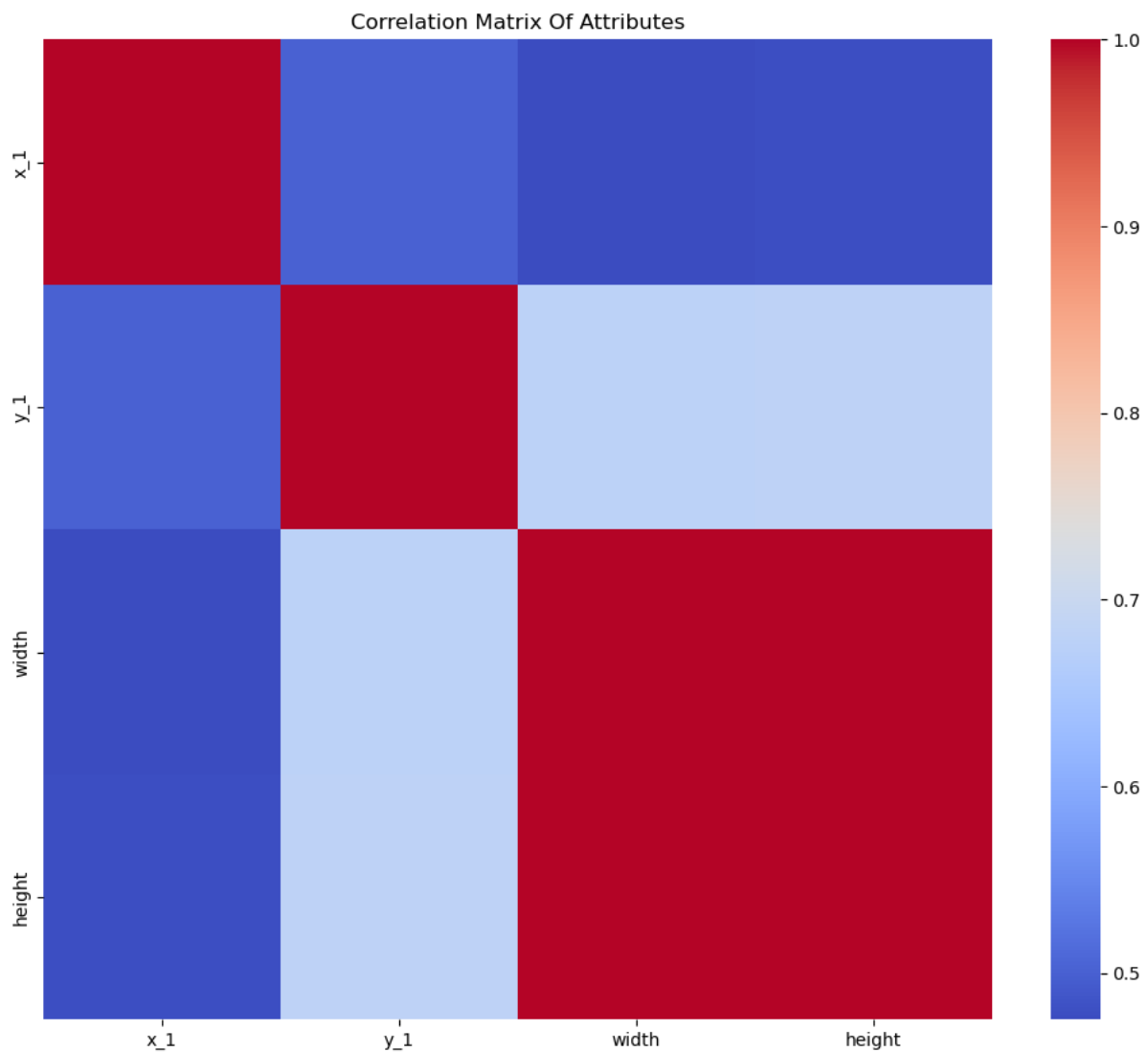
- **Positive Correlation:** There's a strong positive correlation between the width and height of the bounding boxes. This means that as the width of a bounding box increases, its height tends to increase as well.
- **Linearity:** The relationship appears to be approximately linear, suggesting a consistent proportional relationship between width and height.
- **Outliers:** There are a few outliers, which are points that deviate significantly from the general trend. These could be due to objects with unusual aspect ratios or potential errors in the bounding box annotations.
- **Distribution:** Most of the bounding boxes have widths and heights within a certain range, with a few outliers extending the range.

The correlation matrix below shows the relationships between four variables: x_1 , y_1 , width, and height.

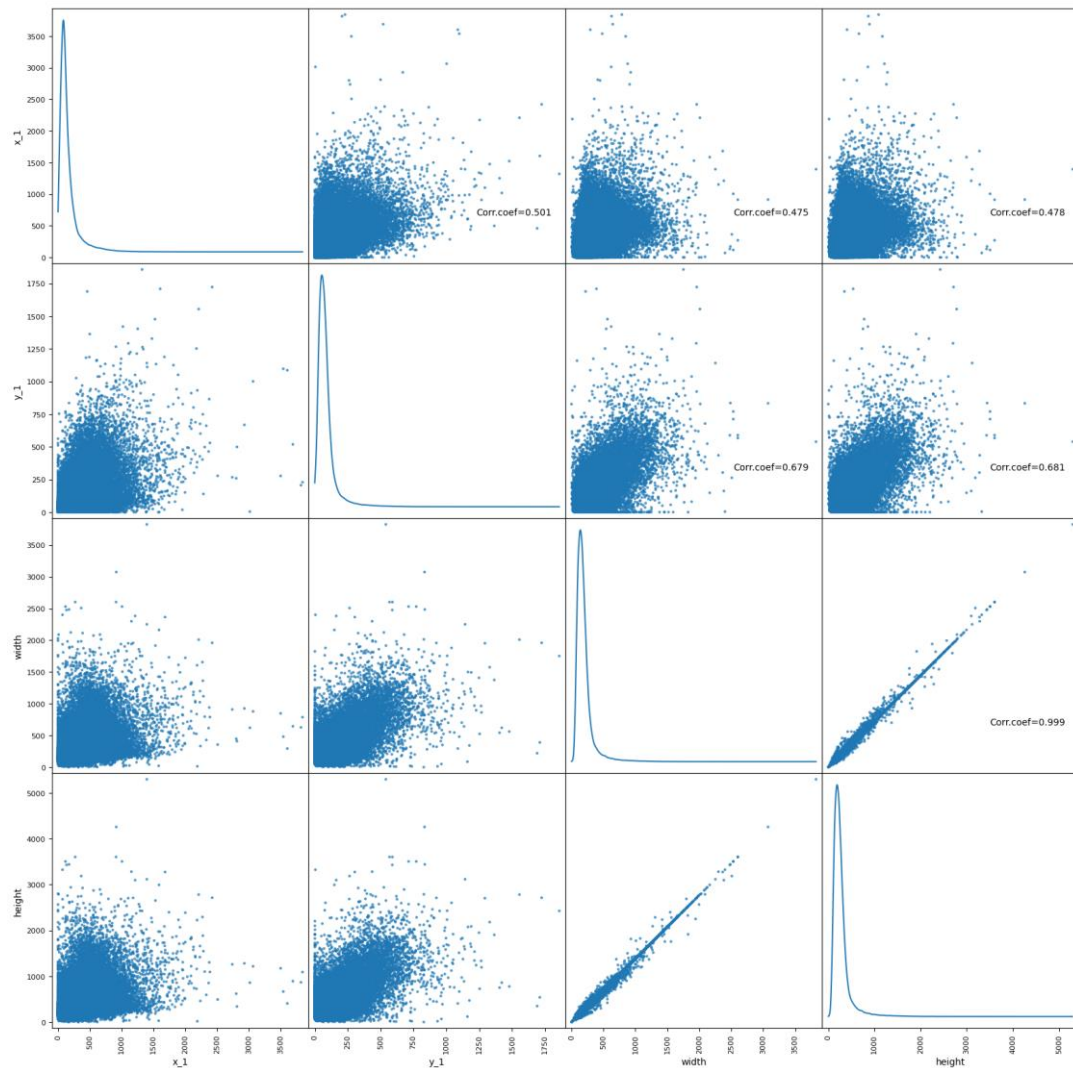
- **Strong Positive Correlations:** The off-diagonal blocks show strong positive correlations between:
 - x_1 and y_1
 - width and height
- **Negative Correlations:** There are no negative correlations between any of the variables.
- **Symmetry:** The correlation matrix is symmetric, meaning the correlation between x_1 and y_1 is the same as the correlation between y_1 and x_1 .

Interpretation:

- The strong positive correlation between x_1 and y_1 indicates that these two variables are highly positively correlated. This means that as one variable increases, the other variable also tends to increase.
- Similarly, the strong positive correlation between width and height suggests that these two variables are also highly positively correlated, indicating that as one variable increases, the other variable tends to increase.
- The absence of negative correlations suggests that none of the variables have a strong inverse relationship with each other.

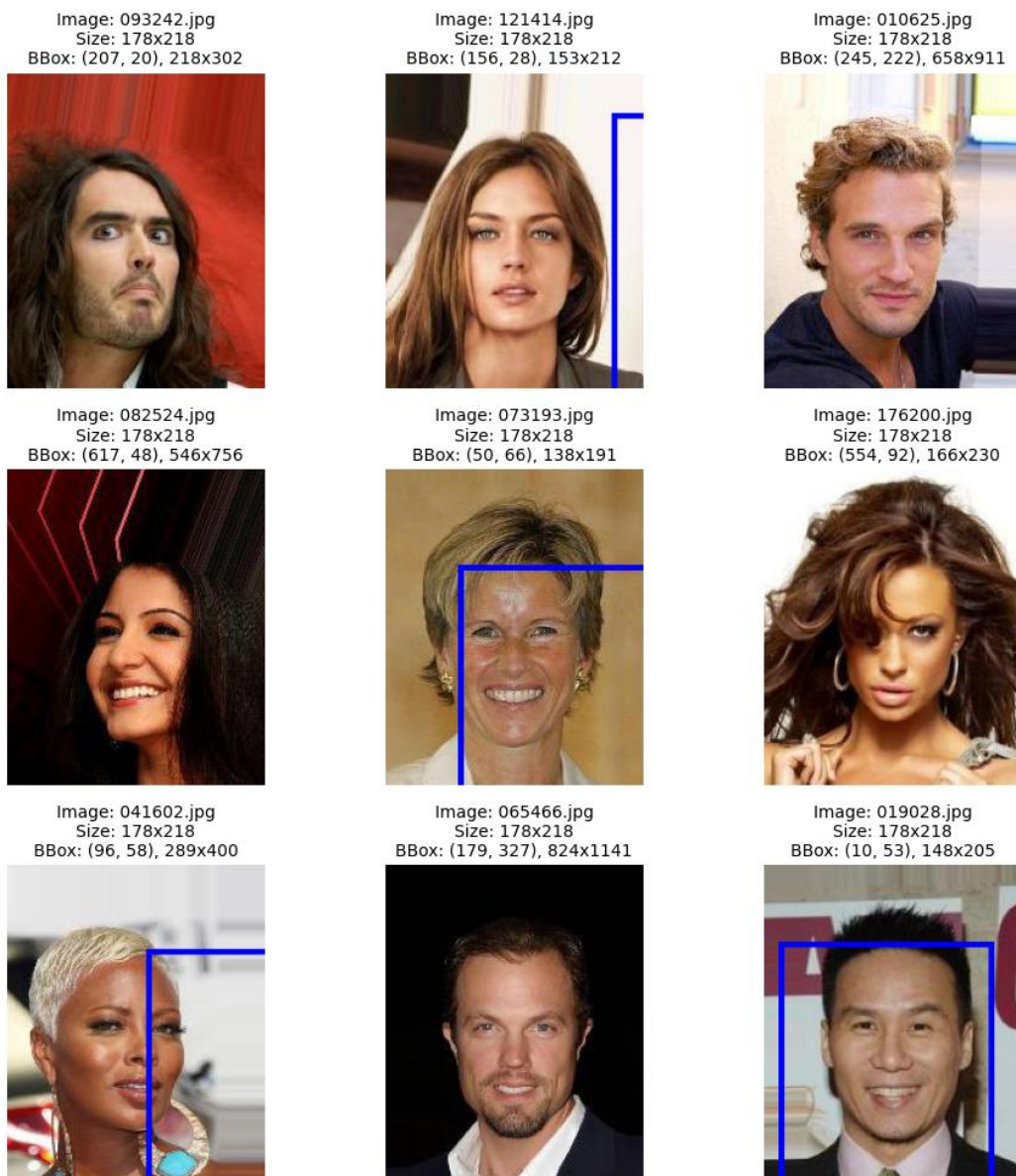


Scatter And Density Plot



- Width and height: The scatter plot for width and height also shows a clear diagonal pattern, indicating a strong positive correlation. This suggests that the width and height of the objects are closely related.
- Other Variables: The scatter plots between x_1 and width and between x_1 and height show a more scattered distribution, suggesting weaker or no correlations.

File list_bbox_celeba.csv has many errors, can't use this file

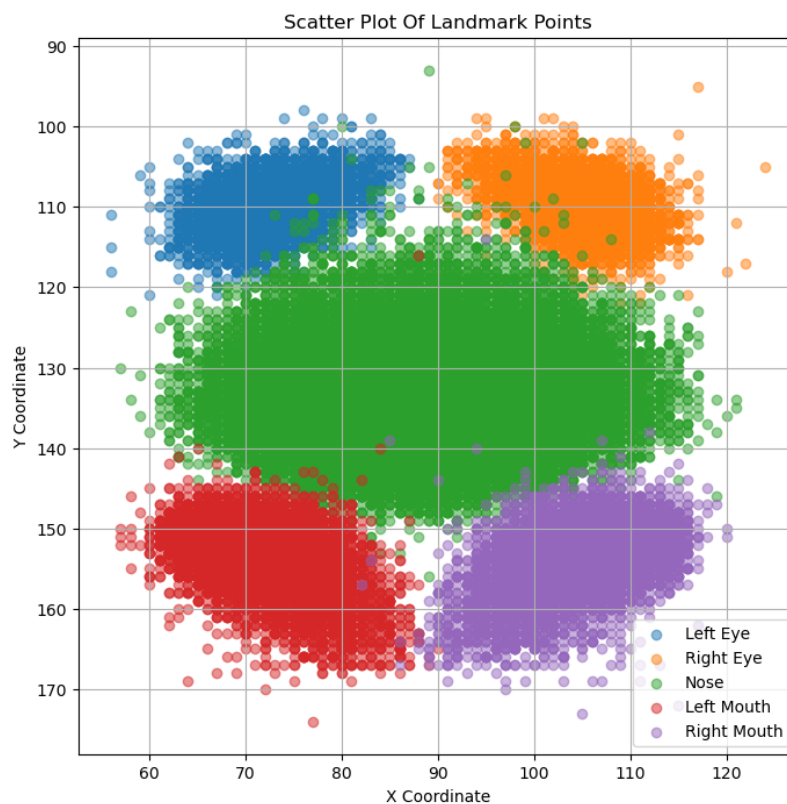
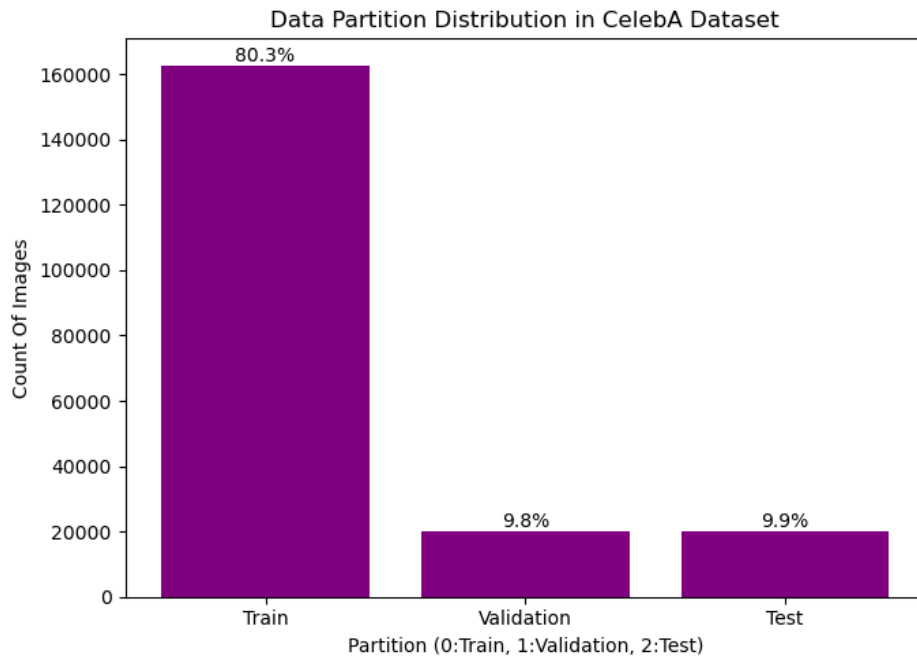


The bar chart shows the distribution of data points among the training, validation, and test sets in the CelebA dataset

Key Observations:

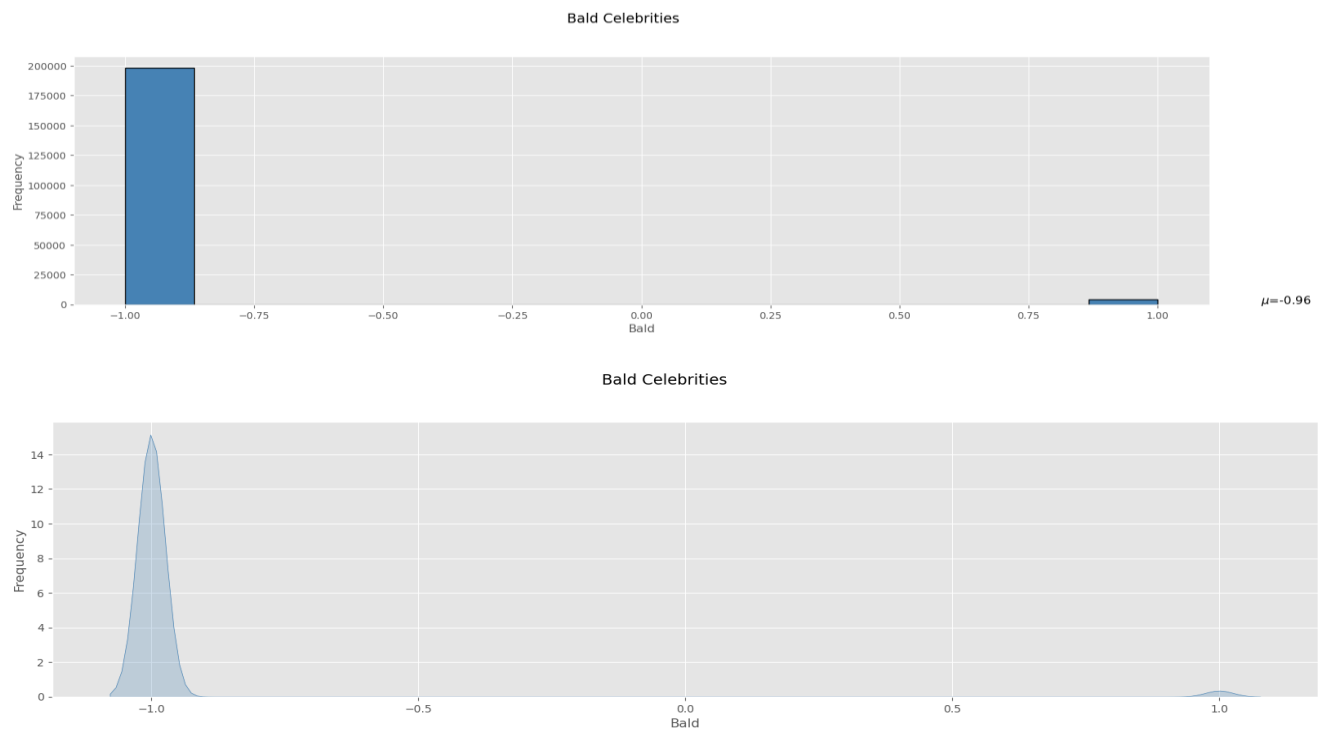
- Data Distribution:
 - The training set comprises much of the data, with 80.3% of the total samples.
 - The validation and test sets each account for 9.8% and 9.9% of the data, respectively.

- **Bar Heights:** The heights of the bars represent the number of images in each partition. The training set bar is significantly taller than the validation and test set bars, reflecting the larger proportion of data allocated to training.
- **Percentages:** The percentages displayed above each bar indicate the proportion of data in each partition relative to the total dataset.



The images are close-up shots of faces, so the landmark points are in fixed areas of the images.

Bald Celebrities



Distribution: The distribution is heavily skewed towards the left, with a large majority of the observations falling within the range of -1.00 to -0.75. There are a few observations around -0.50 and a very small number of observations at 1.00.

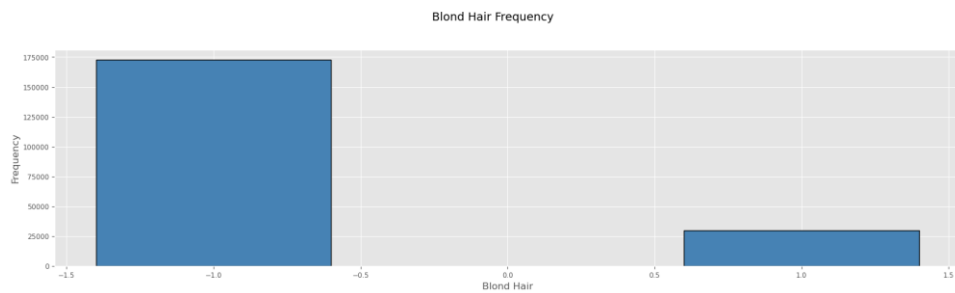
Mean (μ): -0.96

The mean value of the "Bald" variable is -0.96, which suggests that the majority of the celebrities in the dataset are not bald.

Interpretation:

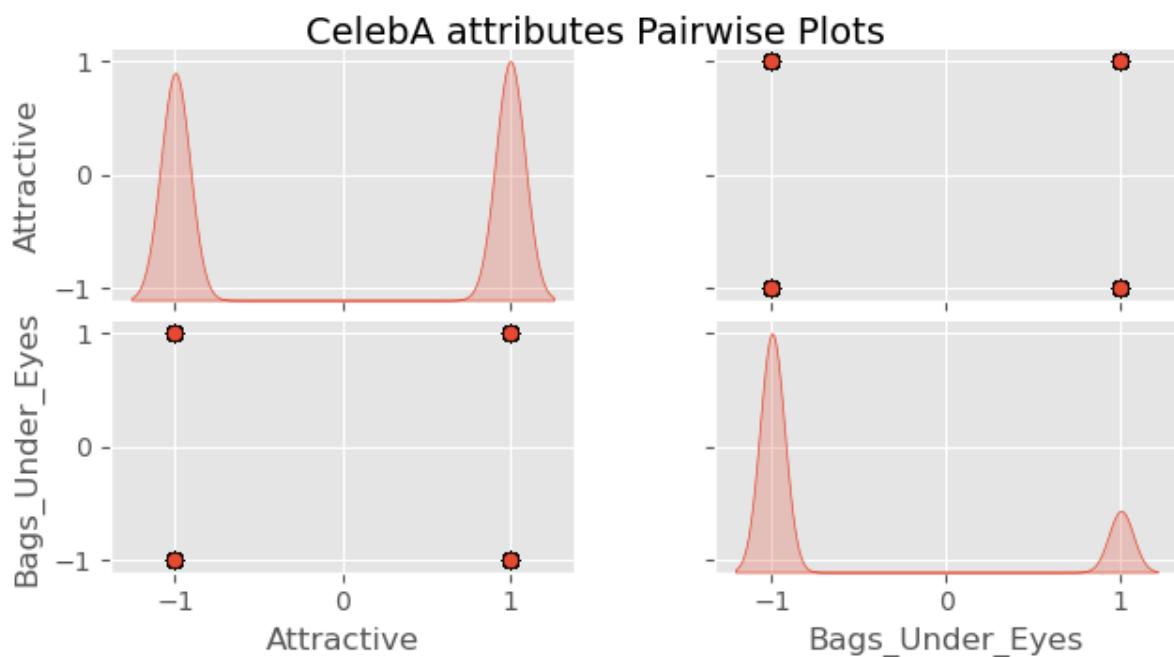
Based on the histogram and the mean value, the dataset likely contains a large number of celebrities who are not bald, with a small proportion of celebrities who are bald. The distribution is highly skewed, indicating that the majority of the observations are clustered towards one end of the scale.

Title: Blond Hair Frequency



Distribution: The distribution is heavily skewed towards the left, with a large majority of the observations falling within the range of -1.00 to -0.75. There are a few observations around -0.50 and a very small number of observations at 1.00.

Title: CelebA attributes Pairwise Plots



Plots:

- Top left: A density plot showing the distribution of the "Attractive" attribute. The x-axis represents the values of "Attractive," and the y-axis represents the density of observations at each value. The distribution is bimodal, with peaks around -1 and 1, suggesting that there are two main groups of individuals in the dataset for attractiveness.
- Top right: A scatter plot showing the relationship between "Attractive" and "Bags_Under_Eyes." The x-axis represents "Attractive," and the y-axis represents "Bags_Under_Eyes." The plot shows that there is a weak positive correlation between the

two attributes, meaning that individuals with higher ratings for "Attractive" tend to also have higher ratings for "Bags_Under_Eyes." However, the correlation is not very strong, as there is a significant amount of scatter in the data.

- Bottom left: A scatter plot showing the relationship between "Bags_Under_Eyes" and "Attractive." This is the same as the top right plot, but with the axes reversed.
- Bottom right: A density plot showing the distribution of the "Bags_Under_Eyes" attribute. The x-axis represents the values of "Bags_Under_Eyes," and the y-axis represents the density of observations at each value. The distribution is also bimodal, with peaks around -1 and 1, suggesting that there are two main groups of individuals in the dataset due to the presence of bags under the eyes.

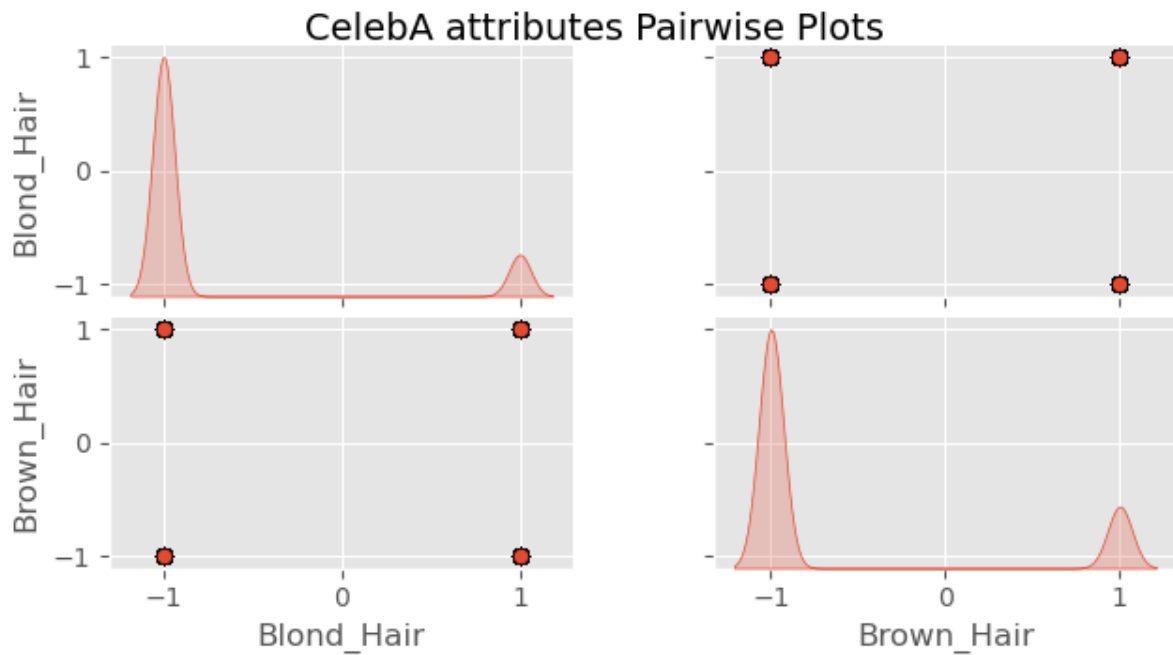
Interpretation:

The pairplot suggests that there are two main groups of individuals in the CelebA dataset concerning both attractiveness and the presence of bags under the eyes. The weak positive correlation between the two attributes indicates that there is a tendency for individuals with higher ratings for attractiveness to also have higher ratings for bags under the eyes, but this relationship is not very strong.

A pairplot of two attributes from the CelebA dataset: "Blond_Hair" and "Brown_Hair."

Interpretation:

The pairplot suggests that hair color is a mutually exclusive attribute in the CelebA dataset. Individuals cannot have both blond and brown hair simultaneously. The bimodal distributions of "Blond_Hair" and "Brown_Hair" indicate that there are two main groups of individuals in the dataset for hair color: those with blond hair and those with brown hair.



Task 3 - Problem Definition (Revisiting Task 1)

CelebA Dataset for Gender Detection Using InceptionV3

This dataset provides a rich resource for training and evaluating machine learning models, particularly those focused on facial recognition and attribute analysis. Here's a breakdown of the key aspects:

Data Composition:

- Images: 202,599 face images, each sized 178x218 pixels.
- Diversity: Images encompass a wide variety of people with diverse ethnicities, ages, and appearances.
- Annotations: Each image is accompanied by:
 - Attribute Labels (40): Binary labels indicating the presence or absence of features like glasses, smiling, or specific hair colors.
 - Landmark Points (5): Coordinates for key facial landmarks like eyes, nose, and mouth.
 - Bounding Boxes: Information about the location and size of each face in the image.

Data Split:

- The dataset offers a pre-defined split for training, validation, and testing sets, facilitating efficient model development and evaluation:
 - Training Set: Images 1-162770 (largest portion for model learning).
 - Validation Set: Images 162771-182637 (used for tuning hyperparameters).
 - Test Set: Images 182638-202599 (assesses model performance on unseen data).

Problem:

The task is to accurately classify the gender of individuals from given facial images. This problem has practical applications in various domains, such as security, healthcare, and entertainment. The CelebA dataset provides a large collection of labeled facial images with diverse genders, making it suitable for training a gender classification model.

Applications:

The CelebA dataset empowers exploration in various areas of computer vision and deep learning:

- Face Detection: Training models to accurately identify faces in images.
 - Facial Attribute Recognition: Classifying facial characteristics like gender, hair color, or expressions.
 - Image Segmentation: Isolating specific facial regions like eyes or lips.
 - Image Generation: Creating realistic facial images or manipulating existing ones
- Accurate Gender Classification: The InceptionV3 model, combined with the CelebA dataset, can achieve high accuracy in gender classification tasks.

Task 4 – Preprocessing

Data Cleaning:

- Loading random Images from the dataset and displaying them



Check for missing values

```
missing_values=df_attr.isnull().sum()
print(missing_values)
```

image_id	0
5_o_Clock_Shadow	0
Arched_Eyebrows	0
Attractive	0
Bags_Under_Eyes	0
Bald	0
Bangs	0
Big_Lips	0
Big_Nose	0
Black_Hair	0
Blond_Hair	0
Blurry	0
Brown_Hair	0
Bushy_Eyebrows	0
Chubby	0
Double_Chin	0
Eyeglasses	0
Goatee	0
Gray_Hair	0
Heavy_Makeup	0
High_Cheekbones	0
Male	0
Mouth_Slightly_Open	0
Mustache	0
Narrow_Eyes	0
No_Beard	0
Oval_Face	0
Pale_Skin	0
Pointy_Nose	0
Receding_Hairline	0
Rosy_Cheeks	0
Sideburns	0
Smiling	0
Straight_Hair	0
Wavy_Hair	0
Wearing_Earrings	0
Wearing_Hat	0
Wearing_Lipstick	0
Wearing_Necklace	0
Wearing_Necktie	0
Young	0
dtype: int64	

Key Observations:

- **No Missing Values:** The output indicates that all columns in df_attr have 0 missing values. This means that there are no null or NaN values present in any of the attributes.
- **Data Type:** The type: int64 at the end suggests that all attributes in the DataFrame are of integer data type.

Interpretation:

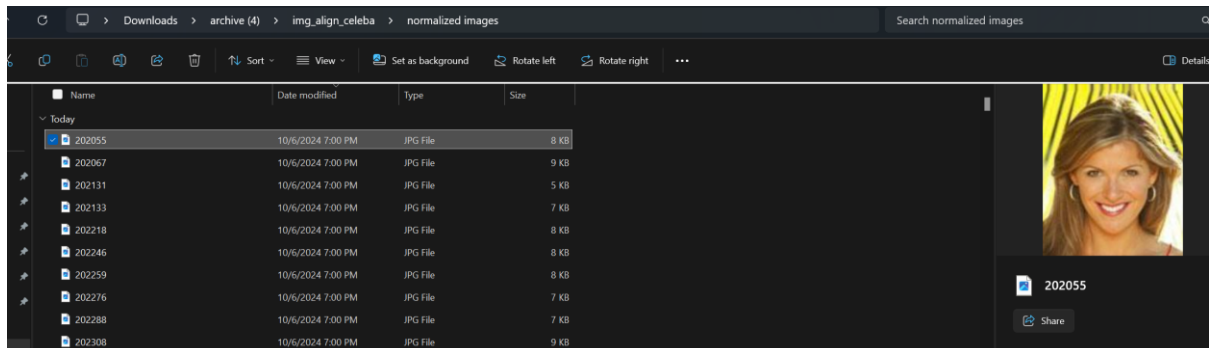
- **Data Quality:** The absence of missing values is a positive sign for data quality. It suggests that the data has been cleaned and prepared appropriately for analysis.

Data Transformation & Normalization:

Copying random 5000 images from the whole dataset to work for further process.

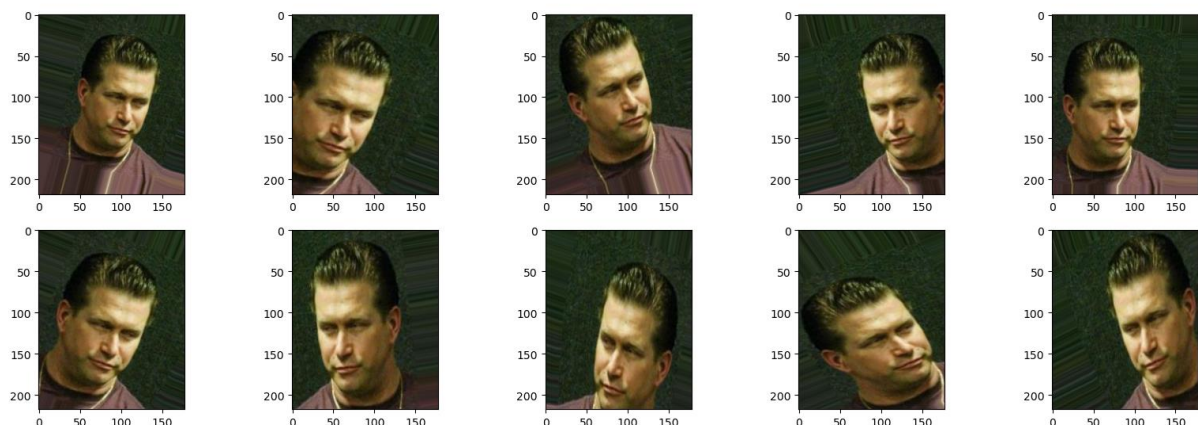
Data Normalization:

Normalizing the images in the dataset and storing them in the normalized images folder in the dataset. Normalization changes the images to be of the same pixel intensity. The normalized images folder contains all the random 5000 normalized images from the cropped folder.



Augmentation: Produced 5 augmented images for every image in the dataset. Augmentation helps recognize facial features recognition as the model inspects images from every angle.

Data Augmentation



Data Augmentation:

Data augmentation is a technique used to artificially increase the size and diversity of a dataset by applying various transformations to existing data. This is done to improve model performance and prevent overfitting.

Transformations:

Based on the numbers in the image, the following transformations have happened:

- Rotation: The numbers along the top axis represent different rotation angles applied to the images.
- Shifting: The numbers along the left axis indicate horizontal or vertical shifts applied to the images.
- Zooming: The combination of both axes represents zooming in or out on the images.
- Flipping: The numbers also indicate horizontal or vertical flipping of the images.

Purpose of Data Augmentation:

Data augmentation is used to:

- Prevent Overfitting: Exposing the model to a wider variety of data helps it generalize better and avoid memorizing specific training examples.
- Improve Model Performance: By introducing additional variations, data augmentation can enhance the model's ability to learn robust features and patterns.

Appropriate Transformations: Images are likely to have variations in orientation, so I applied rotation transformation.

Model Selection, Training, and Optimization

The choice of model - Face recognition

Partitioning the dataset into train, test, and validation images with the help of partition files from the dataset. 0 represents train data, 1 represents validation and 2 represents test data.

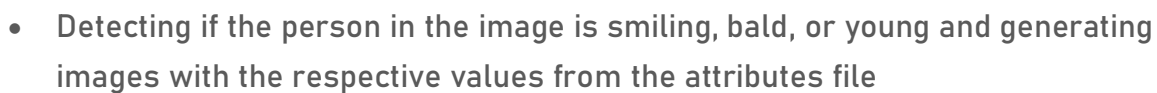
```
partition
0      162770
1      19867
2      19962
Name: count, dtype: int64
```

Using attributes file to know if the person in the image is smiling, bald, young, hair, male or female, etc.

- Facial pointer for eyes, nose, mouth, ears, chin: Face recognition machine learning library for facial pointer recognition.

Facial landmarks are detected correctly for 99% of the images we tested on. It's getting detected even when the face is not in proper alignment.

Image 1 Image 2 Image 3 Image 4 Image 5 Image 6 Image 7 Image 8 Image 9 Image 10 Image 11 Image 12 Image 13 Image 14 Image 15 Image 16 Image 17 Image 18 Image 19 Image 20 Image 21 Image 22 Image 23 Image 24 Image 25



Smiling: 1, Bald: -1, Young: 1

Smiling: -1, Bald: -1, Young: 1

Smiling: 1, Bald: -1, Young: -1

The choice of model-rectifier neural networks

- Here we are using rectifier neural networks. Importing Keras, and TensorFlow to train a model to know if the person in the image is bald or not. Dividing the dataset with partition files to train, validate, and test to train, and test the model to generate output if the person in the image is bald or not. Using SKLearn, pandas, and optimizers.

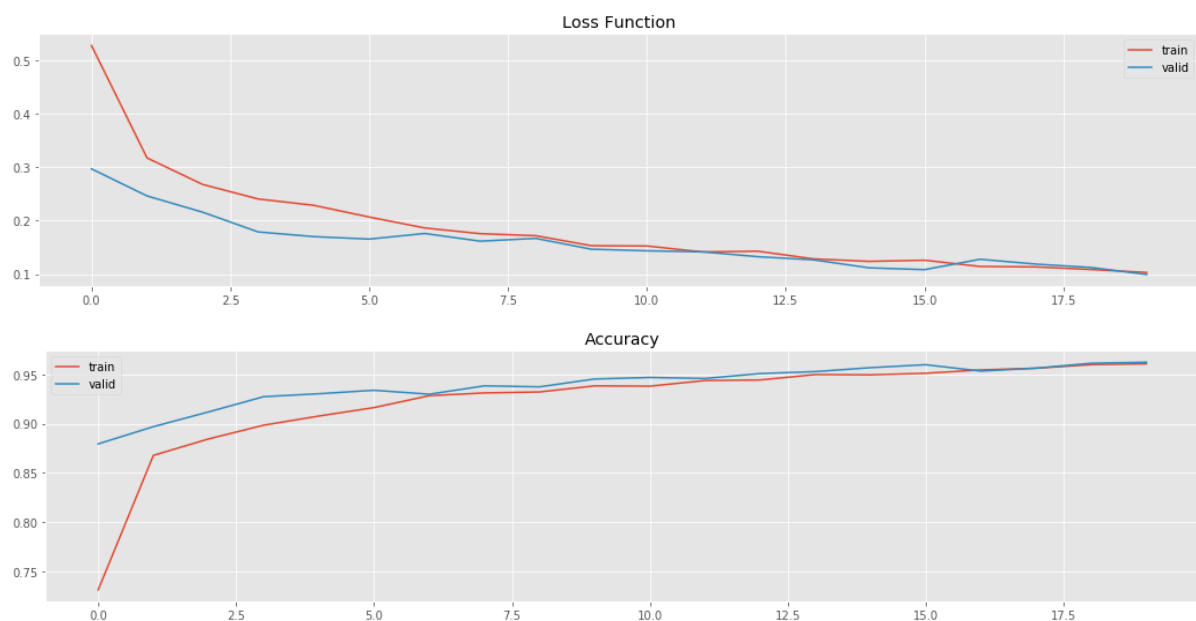
We are using Maxpooling, dropout, density, conv2D, and Relu to train the model. We have achieved 97.7% accuracy to train the model and loss is 1.32 percent. This deep learning model can detect if the person in the model is bald or not with 98 percent accuracy.

accuracy: 0.9777 - loss: 0.1320

The choice of model architecture – InceptionV3

Pre-trained Weights:

- Transfer Learning: InceptionV3 is a pre-trained model on the ImageNet dataset, which contains a vast number of images across various categories. This has learned valuable features from a diverse set of images.
- Efficient Initialization: Using pre-trained weights allows the model to start with a good initial state, accelerating the training process and often leading to better performance.



- Decreasing Loss: Both the training and validation loss are decreasing, which is generally a good sign.

- Gap: There seems to be a slight gap between the training and validation loss curves towards the end of training. While this might indicate some overfitting, it's not a drastic difference.

Model Evaluation

Test accuracy: 94.4500%

f1_score: 0.9422776911076443



Female

91.39% prob.

Real Target: Female

Eyebrows: Not Arched

Bald: No

Young: Yes

Straight Hair: No

Wavy Hair: No

Smiling: No

Eye Bags: No

Nose: Small

The choice of model architecture - DCGAN

Fundamental way GAN Works

GANs (Generative Adversarial Networks) are comprised of two neural networks, a generator and a discriminator, that engage in a competitive game.

- The Generator: This network creates new data instances that are like the training data. In the case of CelebA images, the generator takes a random input (latent vector) and produces a "fake" image that resembles a real CelebA image.
- The Discriminator: This network evaluates the authenticity of images. It classifies images as either "real" (from the training set) or "fake" (generated by the generator). The discriminator outputs a probability between 0 and 1, indicating the likelihood that an image is real.

The Training Process: GAN training is a two-player game where:

1. The Discriminator learns to distinguish real images from fake ones.
2. The Generator learns to create fake images that are increasingly convincing to the discriminator.

The goal is for the generator to become so skilled that it can produce images that the discriminator cannot accurately differentiate from real ones. When this happens, the discriminator's output probability for the generated images should be close to 1.

Convolutional Neural Networks (CNNs) are neural networks with architectural constraints to reduce computational complexity and ensure translational invariance (the network interprets input patterns the same regardless of translation — in terms of image recognition: a banana is a banana regardless of where it is in the image).

Convolutional Neural Networks have three important architectural features.

****Local Connectivity****: Neurons in one layer are only connected to neurons in the next layer that are spatially close to them. This design trims the vast majority of connections between consecutive layers but keeps the ones that carry the most useful information. The assumption made here is that the input data has spatial significance, or in the example of computer vision, the relationship between two distant pixels is probably less significant than two close neighbors.

****Shared Weights****: This is the concept that makes CNNs "convolutional." By forcing the neurons of one layer to share weights, the forward pass (feeding data through the network) becomes the equivalent of convolving a filter over the image to produce a new image. The training of CNNs then becomes the task of learning filters (deciding what features you should look for in the data.)

****Pooling and ReLU****: CNNs have two non-linearities: pooling layers and ReLU functions. Pooling layers consider a block of input data and simply pass on the maximum value. Doing this reduces the size of the output and requires no added parameters to learn, so pooling layers are often used to regulate the size of the network and keep the system below a computational limit. The ReLU function takes one input, x , and returns the maximum of $\{0, x\}$. $\text{ReLU}(x) = \arg\max(x, 0)$. This introduces a similar effect to $\tanh(x)$ or $\text{sigmoid}(x)$ as non-linearities to increase the model's expressive power.

Deep Convolutional GANs (DCGANs)

Deep Convolutional GANs (DCGANs) introduced convolutions to the generator and discriminator networks.

However, this was not simply a matter of adding convolutional layers to the model, since training became even more unstable.

Generator Architecture

The DCGAN generator consists of a series of transposed convolutional layers, each followed by a batch normalization layer and a ReLU activation function. The final layer uses a Tanh activation function.

- **Layer Structure:**
 - The generator has four hidden layers (including the input layer) and an output layer.
 - Transposed convolutional layers are used to upsample the feature maps.
 - Batch normalization helps stabilize training by normalizing the activations.
 - ReLU introduces non-linearity.
- **Layer Parameters:**
 - The 2nd, 3rd, and 4th hidden layers, along with the output layer, have a stride of 2, doubling the image size at each step.
 - The 1st layer has no padding (padding=0), while the others have padding=1 to maintain the same output size.
 - As the image size increases, the number of channels is halved to control the model's complexity.
 - All transposed convolutional layers have a kernel size of 4x4.
- **Output:**
 - The output channel can be 1 for grayscale images or 3 for color images.

This architecture allows the generator to progressively upsample a low-dimensional latent vector into a high-dimensional image, capturing the complex patterns and details of the training data.

The architecture of a discriminator

The discriminator network of a DCGAN consists of 4 hidden layers (again, we treat the input layer as the 1st hidden layer) and 1 output layer. Convolution layers are used in all layers, which are followed by batch normalization layers except that the first layer does not have batch normalization. LeakyReLU activation functions are used in the hidden layers and Sigmoid is used for the output layer. The architecture of the discriminator is shown in the following:

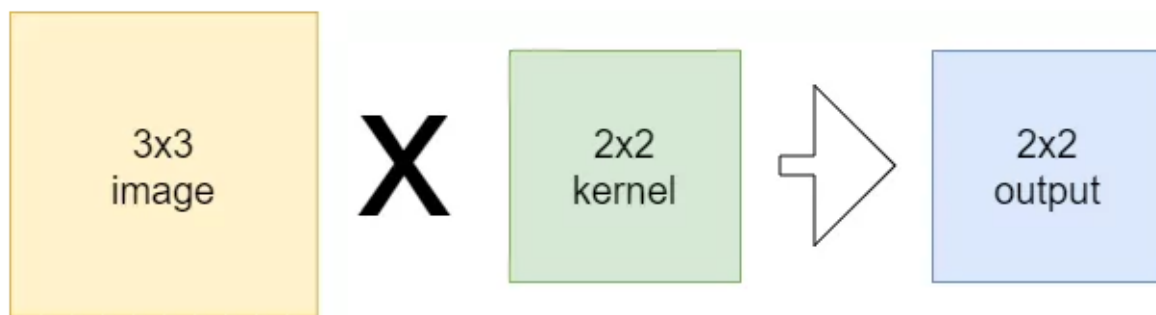
The input channel can be either 1 or 3, depending on whether you are dealing with

grayscale images or color images. All hidden layers have a stride value of 2 and a padding value of 1 so that their output image sizes will be half the input images. As image sizes increase in deeper layers, the number of channels increases by twice. All kernels in convolution layers are of a size of 4 x 4. The output layer has a stride value of 1 and a padding value of 0. It maps 4 x 4 feature maps to single values so that the Sigmoid function can transform the value into prediction confidence.

Several tricks had to be applied to make DCGANs stable and useful:

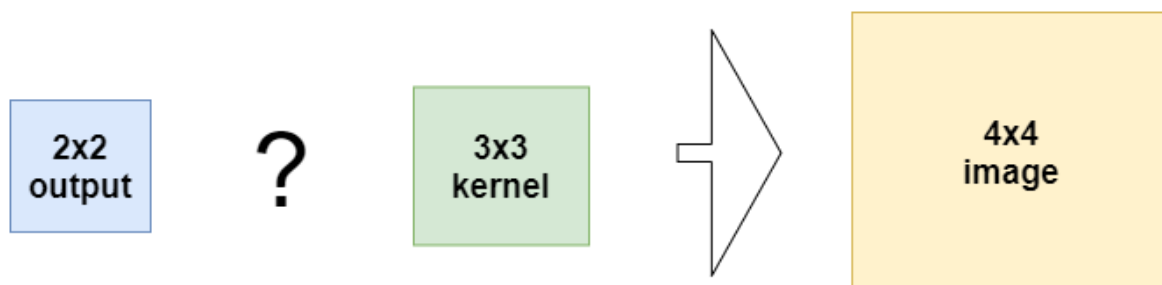
- Batch normalization was applied to both the generator and the discriminator network
- Dropout is used as a regularization technique
- The generator needed a way to upsample the random input vector to an output image. Transposing convolutional layers is employed here
- LeakyRelu and TanH activations are used throughout both networks

Transposed Convolutions



The 2x2 kernel produces a 2x2 output when convolving over a 3x3 image.

But to work in the opposite direction, i.e., to use a smaller input and to learn its larger representation, being the following:



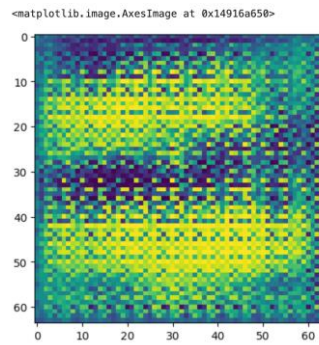
That is where the Transposed convolutions in the Keras API helps.

The Conv2DTranspose layer learns several filters, similar to the regular Conv2D layer.

Remember that the transpose layer simply swaps the backward and forward pass, keeping the rest of the operations the same!

As the transposed convolution will also slide over the input, we must specify a `kernel_size`, as with the normal convolution. Similarly, we must specify `strides`, `output_padding`.

A random noise - It is just plain noise. But the fact that it can create an image from a random noise array proves the model's power.



Generator

Model: "sequential_17"

Layer (type)	Output Shape	Param #
dense_12 (Dense)	(None, 8192)	827,392
reshape_6 (Reshape)	(None, 4, 4, 512)	0
conv2d_transpose_24 (Conv2DTranspose)	(None, 8, 8, 256)	2,097,408
batch_normalization_24 (BatchNormalization)	(None, 8, 8, 256)	1,024
leaky_re_lu_30 (LeakyReLU)	(None, 8, 8, 256)	0
conv2d_transpose_25 (Conv2DTranspose)	(None, 16, 16, 128)	524,416
batch_normalization_25 (BatchNormalization)	(None, 16, 16, 128)	512
leaky_re_lu_31 (LeakyReLU)	(None, 16, 16, 128)	0
conv2d_transpose_26 (Conv2DTranspose)	(None, 32, 32, 64)	131,136
batch_normalization_26 (BatchNormalization)	(None, 32, 32, 64)	256
leaky_re_lu_32 (LeakyReLU)	(None, 32, 32, 64)	0
conv2d_transpose_27 (Conv2DTranspose)	(None, 64, 64, 3)	3,075

Total params: 3,585,219 (13.68 MB)

Trainable params: 3,584,323 (13.67 MB)

Non-trainable params: 896 (3.50 KB)

Discriminator

Model: "sequential_18"

Layer (type)	Output Shape	Param #
conv2d_12 (Conv2D)	(None, 32, 32, 64)	1,792
leaky_re_lu_33 (LeakyReLU)	(None, 32, 32, 64)	0
dropout_12 (Dropout)	(None, 32, 32, 64)	0
conv2d_13 (Conv2D)	(None, 16, 16, 64)	36,928
batch_normalization_27 (BatchNormalization)	(None, 16, 16, 64)	256
leaky_re_lu_34 (LeakyReLU)	(None, 16, 16, 64)	0
dropout_13 (Dropout)	(None, 16, 16, 64)	0
flatten_6 (Flatten)	(None, 16384)	0
dense_13 (Dense)	(None, 1)	16,385

Total params: 55,361 (216.25 KB)

Trainable params: 55,233 (215.75 KB)

Non-trainable params: 128 (512.00 B)

Use of Batch Normalization

Batch normalization standardizes the activations from a prior layer to have a zero mean and unit variance. This has the effect of stabilizing the training process.

Batch normalization limits the amount by which updating the parameters in the previous layers can affect the distribution of inputs received by the current layer. This decreases any unwanted interdependence between parameters across layers, which helps speed up the network training process and increase its robustness, especially when it comes to network parameter initialization.

Batch normalization is used after the activation of convolution in the discriminator model and after transposing convolutional layers in the generator model.

It is added to the model after the hidden layer, but before the activation, such as LeakyReLU.

DCGAN - Combining Generator and Discriminator

Setting discriminator.trainable to False.

Why would we want to do this?

Well, we aren't going to be training the generator model directly — we are going to be combining the generator and discriminator into a single model, then training that. This allows the generator to understand the discriminator so it can update itself more effectively.

Setting discriminator. Trainable to False will only affect the copy of the discriminator in the combined model. This is good! If the copy of the discriminator in the combined model were trainable, it would update itself to be worse at classifying images.

To combine the generator and discriminator, we will be calling the discriminator on the output of the generator.

Final Training Architecture of a DCGAN

DCGAN is super sensitive

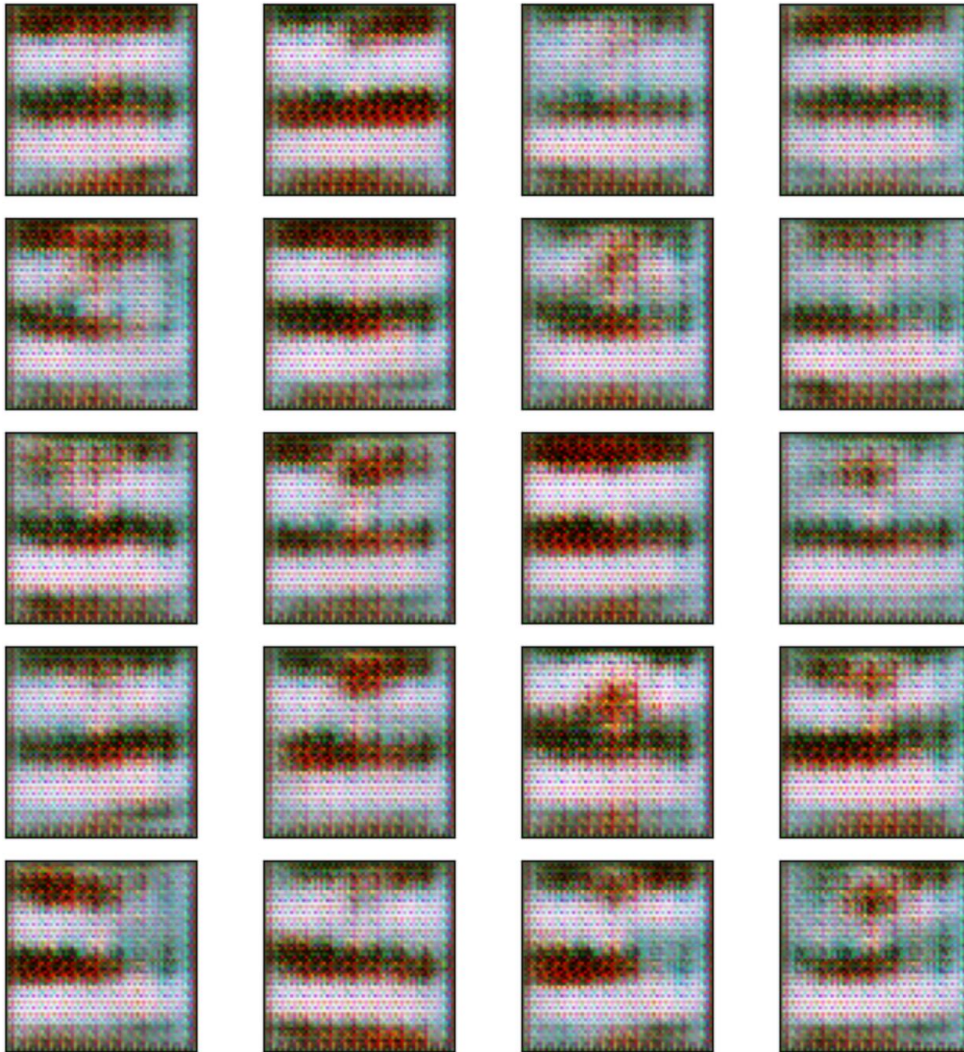
Here, even when we only train a GAN to manipulate 1D data, we must use multiple techniques to ensure stable training. A lot of things could go wrong in the training of GANs. For example, either a generator or a discriminator could overfit if one or the other does not converge. Sometimes, the generator only generates a handful of sample varieties. This is called mode collapse.

It's important that the generator and discriminator do not overpower each other (e.g., that they train at a similar rate).

At the beginning of the training, the generated images will look like random noise. As training progresses, the generated images will look increasingly real just like original celeb-a images. After about 300 epochs, they resemble almost the original.

Predicted Noise

Figure 5.22: 70x70 with 0.1x0.1



Model Evaluation

We are using different measures like F1 measure, Test Accuracy, and loss.

- We compared our model with Haar cascades for extracting facial attributes like mouth, and eyes and it is not working well when the person in the image is not aligned properly.
- We compared the detection of smiling, bald, and young with face recognition model which generates all the attributes regarding the person in the image and the inception v3 model works with 94% accuracy for gender recognition while the other model only works when the attributes are listed in the list of attributes file.